```
26        if (!super.equals(otherObject)) return false;
27        var other = (Manager) otherObject;
28        // super.equals checked that this and other belong to the same class
29        return bonus == other.bonus;
30     }
31
32     public int hashCode()
33     {
34        return java.util.Objects.hash(super.hashCode(), bonus);
35     }
36
37     public String toString()
38     {
39        return super.toString() + "[bonus=" + bonus + "]";
40     }
41  }
```

API java.lang.Object 1.0

- `Class getClass()`
  返回一个类对象，其中包含有关对象的信息。本章稍后会看到 Java 提供了类的运行时表示，封装在 Class 类中。
- `boolean equals(Object otherObject)`
  比较两个对象是否相等，如果两个对象指向相同的存储区域，则这个方法返回 true；否则，返回 false。要在你自己的类中覆盖这个方法。
- `String toString()`
  返回一个字符串，表示这个对象的值。要在你自己的类中覆盖这个方法。

API java.lang.Class 1.0

- `String getName()`
  返回这个类的名字。
- `Class getSuperclass()`
  以 Class 对象的形式返回这个类的超类。

5.3 泛型数组列表

在一些程序设计语言（如 C 或 C++）中，必须在编译时就确定所有数组的大小。程序员对此十分反感，因为这样做将迫使他们做出一些不情愿的折中。例如，一个部门会有多少员工？肯定不会超过 100 人。一旦出现一个拥有 150 名员工的大型部门呢？另外，你愿意为那些仅有 10 名员工的部门浪费 90 个存储空间吗？

在 Java 中，情况就好多了。它允许在运行时确定数组的大小。

```
int actualSize = . . .;
var staff = new Employee[actualSize];
```

当然，这个代码并没有完全解决运行时动态修改数组的问题。一旦确定了数组的大小，就无法再轻松地改变了。在 Java 中，要处理这个常见的情况，可以使用 Java 中的另外一个类，名为 ArrayList。ArrayList 类与数组类似，但在添加或删除元素时，它能够自动地调整容量，而不需要为此额外编写代码。

ArrayList 是一个有类型参数（type parameter）的泛型类（generic class）。为了指定数组列表保存的元素对象的类型，需要用一对尖括号将类名括起来追加到 ArrayList 后面，例如 ArrayList<Employee>。在第 8 章中将看到如何自定义一个泛型类，不过使用 ArrayList 类型并不要求了解它的任何技术细节。

下面几节将介绍如何处理数组列表。

5.3.1 声明数组列表

可以如下声明和构造一个保存 Employee 对象的数组列表：

```
ArrayList<Employee> staff = new ArrayList<Employee>();
```

在 Java 10 中，最好使用 var 关键字以避免重复写类名：

```
var staff = new ArrayList<Employee>();
```

如果没有使用 var 关键字，则可以省略右边的类型参数：

```
ArrayList<Employee> staff = new ArrayList<>();
```

这称为“菱形”语法，因为空尖括号 <> 就像是一个菱形。可以结合 new 操作符使用菱形语法。编译器会检查新值要做什么。如果赋值给一个变量，或传递给某个方法，或者从某个方法返回，编译器会检查这个变量、参数或方法的泛型类型，然后将这个类型放在 <> 中。在这个例子中，new ArrayList<>() 将赋值给一个类型为 ArrayList<Employee> 的变量，所以泛型类型为 Employee。

警告：如果使用 var 声明 ArrayList，就不要使用菱形语法。以下声明

```
var elements = new ArrayList<>();
```

会生成一个 ArrayList<Object>。

注释：Java 5 以前的版本没有提供泛型类，而是有一个保存 Object 类型元素的 ArrayList 类，它是一个“自适应大小”（one-size-fits-all）的集合。你仍然可以使用没有后缀 <. . .> 的 ArrayList。它被认为是一个擦除了类型参数的“原始”类型。

注释：在更老的 Java 版本中，程序员使用 Vector 类实现动态数组。不过，ArrayList 类更加高效，没有任何理由再使用 Vector 类。

使用 add 方法可以将元素添加到数组列表中。例如，下面展示了如何将 Employee 对象添加到一个数组列表中：

```
staff.add(new Employee("Harry Hacker", . . .));
staff.add(new Employee("Tony Tester", . . .));
```

数组列表管理着一个内部的对象引用数组。最终，这个数组的空间有可能全部用尽。这时就显现出数组列表的魅力了：如果调用 add 而内部数组已经满了，数组列表就会自动地创建一个更大的数组，并将所有对象从较小的数组拷贝到较大的数组中。

如果已经知道或能够估计出数组可能存储的元素数量，就可以在填充数组之前调用 ensureCapacity 方法：

```
staff.ensureCapacity(100);
```

这个方法调用将分配一个包含 100 个对象的内部数组。这样一来，前 100 次 add 调用不会带来开销很大的重新分配空间。

另外，还可以把初始容量传递给 ArrayList 构造器：

```
ArrayList<Employee> staff = new ArrayList<>(100);
```

警告： 如下分配数组列表：

```
new ArrayList<>(100) // capacity is 100
```

这与分配一个新数组有所**不同**：

```
new Employee[100] // size is 100
```

数组列表的容量与数组的大小有一个非常重要的区别。如果分配一个有 100 个元素的数组，数组就有 100 个空位置（槽）可以使用。而容量为 100 个元素的数组列表只是**可能**保存 100 个元素（实际上也可以超过 100，不过要以重新分配空间为代价），但是在一开始，甚至完成初始化构造之后，数组列表并不包含任何元素。

size 方法将返回数组列表中包含的实际元素个数。例如，

```
staff.size()
```

将返回 staff 数组列表的当前元素个数，它等价于数组 a 的 a.length。

一旦能够确认数组列表的大小将保持恒定，不再发生变化，就可以调用 trimToSize 方法。这个方法将内存块的大小调整为保存当前元素数量所需要的存储空间。垃圾回收器将回收多余的存储空间。

一旦削减了数组列表的大小，添加新元素就需要再次移动内存块，这很耗费时间，所以应当只有在确认不会再向数组列表添加任何元素时才调用 trimToSize。

C++ 注释： ArrayList 类似于 C++ 的 vector 模板。ArrayList 与 vector 都是泛型类型。但是 C++ 的 vector 模板重载了 [] 操作符以便于访问元素。由于 Java 没有操作符重载，所以必须调用显式的方法。此外，C++ 向量是按值复制。如果 a 和 b 是两个向量，赋值操作 a = b 将会构造一个与 b 长度相同的新向量 a，并将所有的元素由 b 复制到 a。而在 Java 中，这条赋值语句的操作结果是让 a 和 b 引用同一个数组列表。

API java.util.ArrayList<E> 1.2

- ArrayList<E>()

构造一个空数组列表。

- `ArrayList<E>(int initialCapacity)`
  构造一个有指定容量的空数组列表。
- `boolean add(E obj)`
  在数组列表的末尾追加 obj。总是返回 true。
- `int size()`
  返回当前存储在数组列表中的元素个数。(当然，这个值永远不会大于数组列表的容量。)
- `void ensureCapacity(int capacity)`
  确保数组列表在不重新分配内部存储数组的情况下，有足够的容量存储给定数量的元素。
- `void trimToSize()`
  将数组列表的存储容量削减到其当前大小。

5.3.2　访问数组列表元素

很遗憾，天下没有免费的午餐。为了提供数组列表自动扩展容量的便利，这要求使用一种更复杂的语法来访问元素。其原因是 ArrayList 类并不是 Java 程序设计语言的一部分，它只是由某个人编写并放在标准库中的一个实用工具类。

不能使用我们喜爱的 [] 语法格式访问或改变数组的元素，而要使用 get 和 set 方法。

例如，要设置第 i 个元素，可以使用

```
staff.set(i, harry);
```

它等价于对数组 a 的元素赋值（与数组一样，索引值从 0 开始）：

```
a[i] = harry;
```



警告：只有当数组列表的大小大于 i 时，才能够调用 list.set(i,x)。例如，下面这段代码是错误的：

```
var list = new ArrayList<Employee>(100); // capacity 100, size 0
list.set(0, x); // no element 0 yet
```

要使用 add 方法为数组添加新元素，而不是 set 方法，set 方法只是用来替换数组中之前增加的一个元素。

要得到一个数组列表的元素，可以使用

```
Employee e = staff.get(i);
```

这等价于

```
Employee e = a[i];
```



注释：没有泛型类时，原始 ArrayList 类提供的 get 方法别无选择，只能返回 Object。因此，get 方法的调用者必须将返回值强制转换为所需的类型：

```
Employee e = (Employee) staff.get(i);
```

原始 ArrayList 还存在一定的危险性。它的 add 和 set 方法接受任意类型的对象。对于下面这个调用

```
staff.set(i, "Harry Hacker");
```

它能正常编译而不会给出任何警告，只有在获取对象并试图对它进行强制类型转换时，才会发现有问题。如果使用 ArrayList<Employee>，编译器就会检测到这个错误。

下面这个技巧可以一举两得，既可以灵活地扩展数组，又可以方便地访问数组元素。首先，创建一个数组列表，并添加所有的元素：

```
var list = new ArrayList<X>();
while (. . .)
{
   x = . . .;
   list.add(x);
}
```

执行完上述操作后，使用 toArray 方法将数组元素复制到一个数组中：

```
var a = new X[list.size()];
list.toArray(a);
```

有时需要在数组列表的中间增加元素，为此可以使用 add 方法并提供一个索引参数：

```
int n = staff.size() / 2;
staff.add(n, e);
```

位置 n 及之后的所有元素都要向后移动一个位置，为新元素留出空间。插入新元素后，如果数组列表的新大小超过了容量，数组列表就会重新分配它的存储数组。

类似地，可以从数组列表中间删除一个元素：

```
Employee e = staff.remove(n);
```

位于这个位置之后的所有元素都向前移动一个位置，并且数组的大小减 1。

插入和删除元素的效率很低。对于较小的数组列表来说，不必担心这个问题。但如果存储的元素很多，又经常需要在中间插入和删除元素，就应该考虑使用链表了。有关如何用链表编程的内容将在第 9 章介绍。

可以使用“for each”循环遍历数组列表的内容：

```
for (Employee e : staff)
   do something with e
```

这个循环和以下代码具有相同的效果：

```
for (int i = 0; i < staff.size(); i++)
{
   Employee e = staff.get(i);
   do something with e
}
```

程序清单 5-7 对第 4 章中的 EmployeeTest 做了修改。在这里，将 Employee[] 数组替换成了

ArrayList<Employee>。请注意下面的变化：

- 不必指定数组的大小。
- 使用 add 增加任意多个元素。
- 使用 size() 而不是 length 统计元素个数。
- 使用 a.get(i) 而不是 a[i] 来访问元素。

程序清单 5-7　arrayList/ArrayListTest.java

```
1  package arrayList;
2
3  import java.util.*;
4
5  /**
6   * This program demonstrates the ArrayList class.
7   * @version 1.11 2012-01-26
8   * @author Cay Horstmann
9   */
10 public class ArrayListTest
11 {
12    public static void main(String[] args)
13    {
14       // fill the staff array list with three Employee objects
15       var staff = new ArrayList<Employee>();
16
17       staff.add(new Employee("Carl Cracker", 75000, 1987, 12, 15));
18       staff.add(new Employee("Harry Hacker", 50000, 1989, 10, 1));
19       staff.add(new Employee("Tony Tester", 40000, 1990, 3, 15));
20
21       // raise everyone's salary by 5%
22       for (Employee e : staff)
23          e.raiseSalary(5);
24
25       // print out information about all Employee objects
26       for (Employee e : staff)
27          System.out.println("name=" + e.getName() + ",salary=" + e.getSalary() + ",hireDay="
28             + e.getHireDay());
29    }
30 }
```

API java.util.ArrayList<E> 1.2

- E set(int index, E obj)

  将值 obj 放置在数组列表的指定索引位置，返回之前的内容。

- E get(int index)

  得到指定索引位置存储的值。

- void add(int index, E obj)

  后移元素从而将 obj 插入指定索引位置。

- E remove(int index)

  删除指定索引位置的元素，并将后面的所有元素前移。返回所删除的元素。

第9章 集 合

- ▲ Java 集合框架
- ▲ 集合框架中的接口
- ▲ 具体集合
- ▲ 映射
- ▲ 副本与视图
- ▲ 算法
- ▲ 遗留的集合

以自然的方式实现方法或者非常关注性能时，你选择的不同数据结构会带来很大差异。是否需要快速地搜索成千上万（甚至上百万）个有序的数据项？是否需要快速地在有序序列中间插入元素或删除元素？是否需要在键与值之间建立关联？

本章将介绍如何利用 Java 类库帮助我们实现程序设计所需的传统数据结构。在大学的计算机科学课程中，有一门数据结构（Data Structure）课程，通常要讲授一个学期，因此，有许许多多专门探讨这个重要主题的书籍。与大学课程所讲述的内容不同，这里将跳过理论部分，仅介绍如何使用标准库中的集合类。

9.1 Java 集合框架

Java 最初的版本只为最常用的数据结构提供了很少的一组类：Vector、Stack、Hashtable、BitSet 与 Enumeration 接口，其中 Enumeration 接口提供了一种抽象机制，用于访问任意容器中的元素。这是一个很明智的选择，要想建立一个全面的集合类库，这需要大量的时间和高超的技能。

随着 Java 1.2 的问世，设计人员感到是时候推出一组功能完备的数据结构了。面对一大堆相互冲突的设计问题，他们希望让类库规模很小而且要易于学习，不希望像 C++ 的“标准模板库”（即 STL）那样复杂，但又希望能够得到 STL 率先提出的“泛型算法”所具有的优点。他们还希望遗留的类能融入这个新框架。与集合类库的所有设计者一样，他们必须做出一些艰难的选择，于是，在这个过程中，他们做出了一些独具特色的设计决定。这一节将介绍 Java 集合框架的基本设计，展示如何具体使用，并解释一些颇具争议的特性背后的考虑。

9.1.1 集合接口与实现分离

与现代的数据结构类库的常见做法一样，Java 集合类库也将接口（interface）与实现（implementation）分离。下面利用我们熟悉的一个数据结构——队列（queue）来说明接口与实现如何分离。

队列接口（queue interface）指出可以在队尾添加元素，在队头删除元素，并且可以查找

队列中元素的个数。当需要收集对象并按照“先进先出”方式获取对象时，就应该使用队列（见图 9-1）。

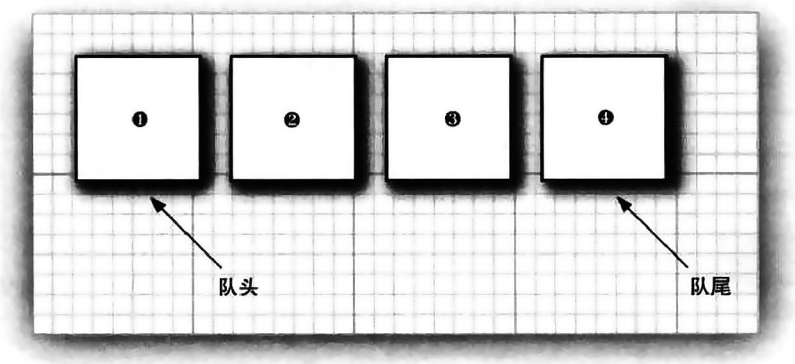


图 9-1 队列

队列接口的最简形式可能如下所示：

```
public interface Queue<E> // a simplified form of the interface in the standard library
{
   void add(E element);
   E remove();
   int size();
}
```

这个接口并没有说明队列是如何实现的。队列通常有两种实现方式：一种是使用循环数组；另一种是使用链表（见图 9-2）。

每个实现都可以用一个实现了 Queue 接口的类表示。

```
public class CircularArrayQueue<E> implements Queue<E> // not an actual library class
{
   private int head;
   private int tail;

   CircularArrayQueue(int capacity) { . . . }
   public void add(E element) { . . . }
   public E remove() { . . . }
   public int size() { . . . }
   private E[] elements;
}

public class LinkedListQueue<E> implements Queue<E> // not an actual library class
{
   private Link head;
   private Link tail;

   LinkedListQueue() { . . . }
   public void add(E element) { . . . }
   public E remove() { . . . }
   public int size() { . . . }
}
```

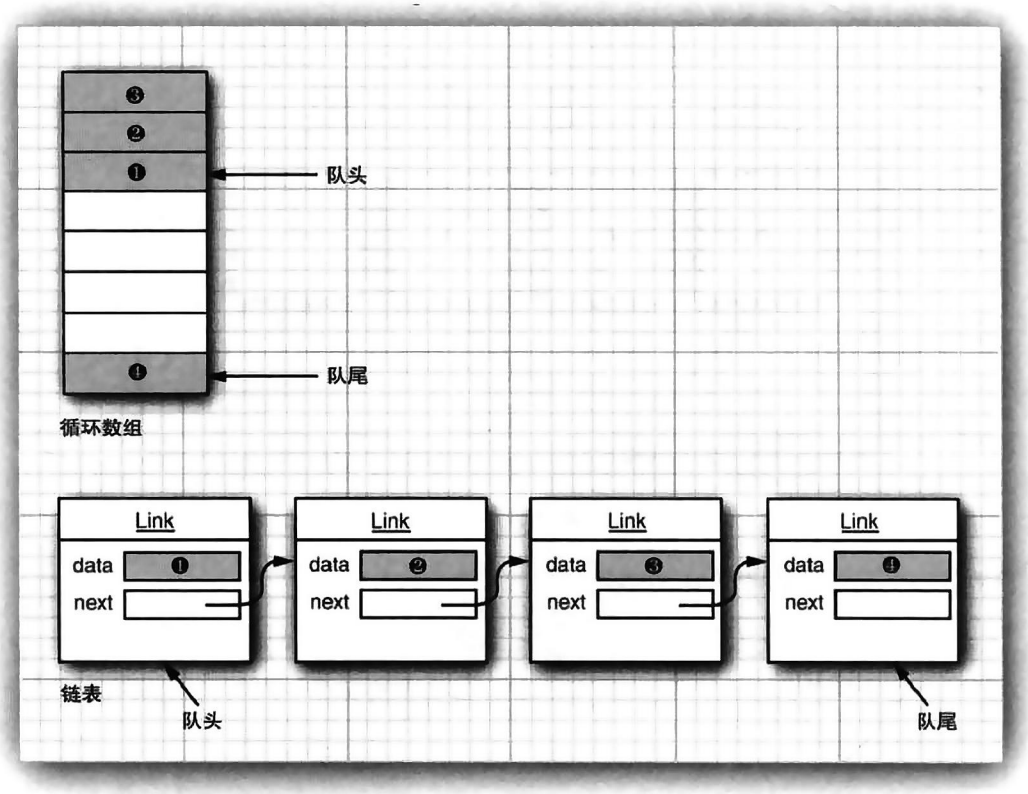


图 9-2 队列的实现

注释： 实际上，Java 类库没有名为 CircularArrayQueue 和 LinkedListQueue 的类。这里只是以这些类为例来解释集合接口与实现在概念上的区分。如果需要一个循环数组队列，可以使用 ArrayDeque 类。如果需要一个链表队列，就直接使用 LinkedList 类，这个类实现了 Queue 接口。

在程序中使用队列时，一旦已经构造了集合，你不需要知道究竟使用了哪种实现。因此，只是在构造集合对象时，才会使用具体的类。可以使用接口类型（interface type）存放集合引用。

```
Queue<Customer> expressLane = new CircularArrayQueue<>(100);
expressLane.add(new Customer("Harry"));
```

利用这种方法，一旦改变了想法，你可以很轻松地使用另外一种不同的实现。只需要修改程序中的一个地方（即调用构造器的语句）。如果觉得 LinkedListQueue 是个更好的选择，就将代码修改为：

```
Queue<Customer> expressLane = new LinkedListQueue<>();
expressLane.add(new Customer("Harry"));
```

为什么选择这种实现，而不选择其他实现呢？接口本身并不能说明一种实现的效率如何。某种程度上讲，循环数组要比链表更高效，因此多数人优先选择循环数组。不过，通常

来讲，这样做也需要付出一定的代价。

循环数组是一个有界（bounded）集合，它的容量有限。如果程序中要收集的对象数量没有上限，就最好使用链表实现。

在研究 API 文档时，会发现另外一组名字以 Abstract 开头的类，例如，AbstractQueue。这些类是为类库实现者而设计的。如果想要实现自己的队列类（也许不太可能），会发现扩展 AbstractQueue 类要比实现 Queue 接口中的所有方法更容易。

9.1.2 Collection 接口

在 Java 类库中，集合类的基本接口是 Collection 接口。这个接口有两个基本方法：

```
public interface Collection<E>
{
   boolean add(E element);
   Iterator<E> iterator();
   . . .
}
```

除了这两个方法之外，还有几个方法，稍后将会介绍。

add 方法用于向集合中添加元素。如果添加元素确实改变了集合就返回 true；如果集合没有发生变化就返回 false。例如，如果试图向集（set）中添加一个对象，而这个对象在集中已经存在，这个 add 请求就没有实效，因为集中不允许有重复的对象。

iterator 方法用于返回一个实现了 Iterator 接口的对象。可以使用这个迭代器对象依次访问集合中的元素。下一节讨论迭代器。

9.1.3 迭代器

Iterator 接口包含 4 个方法：

```
public interface Iterator<E>
{
   E next();
   boolean hasNext();
   void remove();
   default void forEachRemaining(Consumer<? super E> action);
}
```

通过反复调用 next 方法，可以逐个访问集合中的每个元素。但是，如果到达了集合的末尾，next 方法将抛出一个 NoSuchElementException。因此，在调用 next 之前需要调用 hasNext 方法。如果迭代器对象还有更多可以访问的元素，这个方法就返回 true。如果想要查看集合中的所有元素，就请求一个迭代器，当 hasNext 返回 true 时就反复地调用 next 方法。例如：

```
Collection<String> c = . . .;
Iterator<String> iter = c.iterator();
while (iter.hasNext())
{
   String element = iter.next();
```

- void clear()
  从这个集合中删除所有元素。
- boolean retainAll(Collection<?> other)
  从这个集合中删除所有与 other 集合中元素不同的元素。如果由于这个调用改变了集合，返回 true。
- Object[] toArray()
  返回这个集合中的对象的数组。
- <T> T[] toArray(IntFunction<T[]> generator) **11**
  返回这个集合中的对象的数组。这个数组用 generator 构造，这通常是一个构造器表达式 T[]::new。

API ***java.util.Iterator<E>* 1.2**

- boolean hasNext()
  如果存在另一个可访问的元素，返回 true。
- E next()
  返回将要访问的下一个对象。如果已经到达了集合的末尾，将抛出一个 NoSuchElementException。
- void remove()
  删除上次访问的对象。这个方法必须紧跟在访问一个元素之后。如果访问上一个元素之后集合已经发生了变化，这个方法将抛出一个 IllegalStateException。
- default void forEachRemaining(Consumer<? super E> action) **8**
  访问元素，并传递到指定的动作，直到再没有更多元素，或者这个动作抛出一个异常。

9.2 集合框架中的接口

Java 集合框架为不同类型的集合定义了大量接口，如图 9-4 所示。

集合有两个基本接口：Collection 和 Map。我们已经看到，可以用以下方法在集合中插入元素：

```
boolean add(E element)
```

不过，映射包含键 / 值对，要用 put 方法在映射中插入元素：

```
V put(K key, V value)
```

要从集合读取元素，可以用迭代器访问元素。不过，可以使用 get 方法从映射中读取值：

```
V get(K key)
```

List 是一个有序集合（ordered collection）。元素会增加到容器中的特定位置。可以采用两种方式访问元素：使用迭代器访问，或者使用一个整数索引来访问。后面这种方法称为随机访问（random access），因为这样可以按任意顺序访问元素。与之不同，使用迭代器访问时，必须顺序地访问元素。

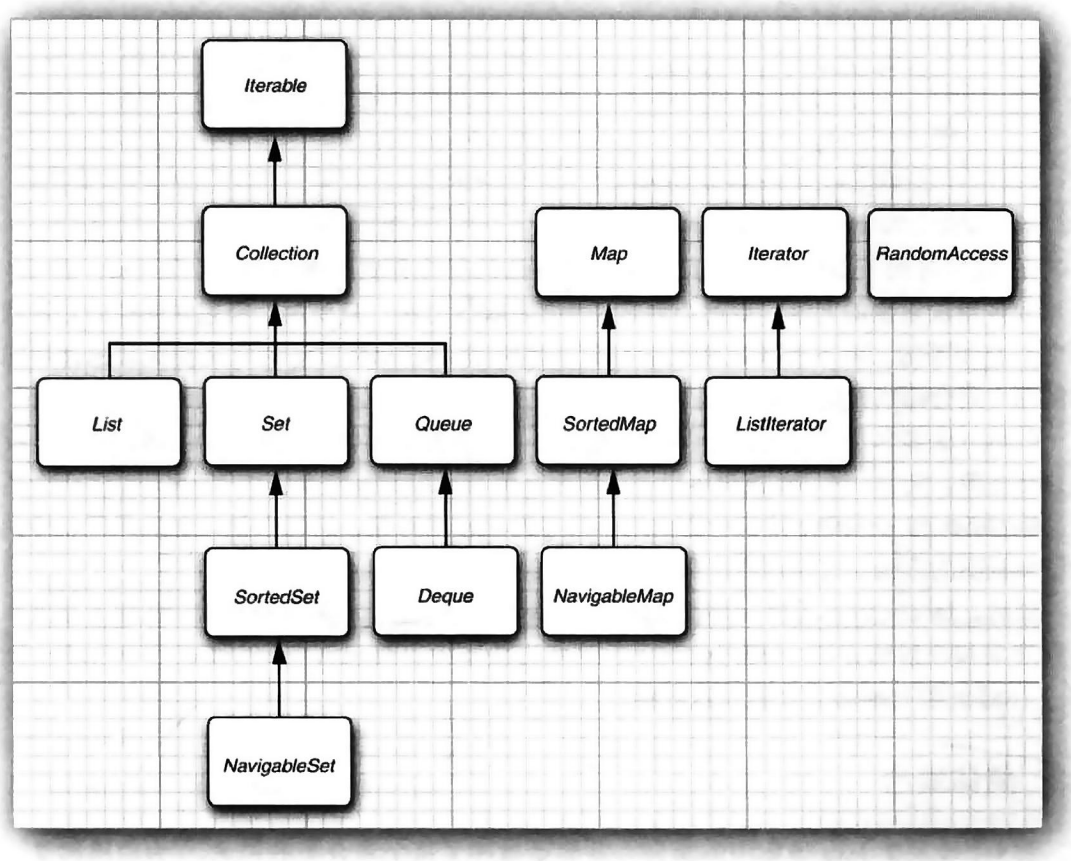


图 9-4 集合框架的接口

List 接口定义了多个用于随机访问的方法：

```
void add(int index, E element)
void remove(int index)
E get(int index)
E set(int index, E element)
```

ListIterator 接口是 Iterator 的一个子接口。它定义了一个方法用于在迭代器位置前面增加一个元素：

```
void add(E element)
```

坦率地讲，集合框架的这个方面设计得很不好。实际上有两种有序集合，其性能开销有很大差异。由数组支持的有序集合可以快速地随机访问，因此适合使用 List 方法并提供一个整数索引来访问。与之不同，链表尽管也是有序的，但是随机访问很慢，所以最好使用迭代器来遍历。如果原先提供两个接口就会容易一些了。

注释：为了避免对链表执行随机访问操作，Java 1.4 引入了一个标记接口 RandomAccess。这个接口不包含任何方法，不过可以用它来测试一个特定的集合是否支持高效的随机访问：

```
 * This program demonstrates the use of a priority queue.
 * @version 1.02 2015-06-20
 * @author Cay Horstmann
 */
public class PriorityQueueTest
{
   public static void main(String[] args)
   {
      var pq = new PriorityQueue<LocalDate>();
      pq.add(LocalDate.of(1906, 12, 9)); // G. Hopper
      pq.add(LocalDate.of(1815, 12, 10)); // A. Lovelace
      pq.add(LocalDate.of(1903, 12, 3)); // J. von Neumann
      pq.add(LocalDate.of(1910, 6, 22)); // K. Zuse

      System.out.println("Iterating over elements . . .");
      for (LocalDate date : pq)
         System.out.println(date);
      System.out.println("Removing elements . . .");
      while (!pq.isEmpty())
         System.out.println(pq.remove());
   }
}
```

API java.util.PriorityQueue 5

- `PriorityQueue()`
- `PriorityQueue(int initialCapacity)`
  构造一个存放 Comparable 对象的优先队列。
- `PriorityQueue(int initialCapacity, Comparator<? super E> c)`
  构造一个优先队列，并使用指定的比较器对元素进行排序。

9.4 映射

作为一个集合，集允许你快速地查找现有的元素。但是，要查找一个元素，需要有所查找的那个元素的准确副本。这不是一种常见的查找方式。通常，我们知道某些关键信息，希望查找与之关联的元素。映射（map）数据结构就是为此设计的。映射用来存放键 / 值对。如果提供了键，可以查找一个值。例如，可以存储一个员工记录表，其中键为员工 ID，值为 Employee 对象。在下面的小节中，我们会学习如何使用映射。

9.4.1 基本映射操作

Java 类库为映射提供了两个通用的实现：HashMap 和 TreeMap。这两个类都实现了 Map 接口。

散列映射对键进行散列，树映射根据键的顺序将它们组织为一个搜索树。散列或比较函数只应用于键。与键关联的值不进行散列或比较。

应该选择散列映射还是树映射呢？与集一样，散列稍微快一些，如果不需要按照有序的

顺序访问键，最好选择散列映射。

可以如下建立一个散列映射来存储员工信息：

```
var staff = new HashMap<String, Employee>(); // HashMap implements Map
var harry = new Employee("Harry Hacker");
staff.put("987-98-9996", harry);
. . .
```

每当向映射中添加一个对象时，必须同时提供一个键。在这里，键是一个字符串，对应的值是 Employee 对象。

要获取一个对象，必须使用键（因此必须记住键）。

```
var id = "987-98-9996";
Employee e = staff.get(id); // gets harry
```

如果映射中没有存储与指定键对应的信息，get 将返回 null。

null 返回值可能并不方便。有时对于没有出现在映射中的键，可以有一个合适的默认值。然后使用 getOrDefault 方法。

```
Map<String, Integer> scores = . . .;
int score = scores.getOrDefault(id, 0); // gets 0 if the id is not present
```

键必须是唯一的。不能对同一个键存放两个值。如果用同一个键调用两次 put 方法，第二个值就会取代第一个值。实际上，put 将返回与这个键参数关联的上一个值。

remove 方法从映射中删除给定键对应的元素。size 方法返回映射中的元素数。

要迭代处理映射的键和值，最容易的方法是使用 forEach 方法。可以提供一个接收键和值的 lambda 表达式。映射中的每一项会依序调用这个表达式。

```
scores.forEach((k, v) ->
   System.out.println("key=" + k + ", value=" + v));
```

程序清单 9-6 显示了映射的具体使用。首先将键 / 值对添加到映射中。然后，从映射中删除一个键，同时与之关联的值也会删除。接下来，修改与某一个键关联的值，并调用 get 方法查找一个值。最后，迭代处理元素集。

程序清单 9-6　map/MapTest.java

```
1  package map;
2
3  import java.util.*;
4
5  /**
6   * This program demonstrates the use of a map with key type String and value type Employee.
7   * @version 1.12 2015-06-21
8   * @author Cay Horstmann
9   */
10 public class MapTest
11 {
12    public static void main(String[] args)
13    {
14       var staff = new HashMap<String, Employee>();
15       staff.put("144-25-5464", new Employee("Amy Lee"));
```

```
16        staff.put("567-24-2546", new Employee("Harry Hacker"));
17        staff.put("157-62-7935", new Employee("Gary Cooper"));
18        staff.put("456-62-5527", new Employee("Francesca Cruz"));
19
20        // print all entries
21
22        System.out.println(staff);
23
24        // remove an entry
25
26        staff.remove("567-24-2546");
27
28        // replace an entry
29
30        staff.put("456-62-5527", new Employee("Francesca Miller"));
31
32        // look up a value
33
34        System.out.println(staff.get("157-62-7935"));
35
36        // iterate through all entries
37
38        staff.forEach((k, v) ->
39           System.out.println("key=" + k + ", value=" + v));
40     }
41  }
```

API *java.util.Map<K, V>* **1.2**

- `V get(Object key)`
  获得与键关联的值；返回与键关联的对象，或者如果在映射中没有找到这个键，则返回 null。实现类可能禁止键为 null。
- `default V getOrDefault(Object key, V defaultValue)`
  获得与键关联的值；返回与键关联的对象，或者如果在映射中没有找到这个键，则返回 defaultValue。
- `V put(K key, V value)`
  将关联的一对键和值放到映射中。如果这个键已经存在，新对象将取代之前与这个键关联的对象。这个方法将返回键对应的旧值。如果之前没有这个键，则返回 null。实现类可能禁止键或值为 null。
- `void putAll(Map<? extends K, ? extends V> entries)`
  将指定映射中的所有映射条目添加到这个映射中。
- `boolean containsKey(Object key)`
  如果映射中有这个键，返回 true。
- `boolean containsValue(Object value)`
  如果映射中有这个值，返回 true。
- `default void forEach(BiConsumer<? super K,? super V> action)` **8**

对这个映射中的所有键 / 值对应用这个动作。

API java.util.HashMap<K, V> 1.2

- HashMap()
- HashMap(int initialCapacity)
- HashMap(int initialCapacity, float loadFactor)

构造一个空散列映射，它具有指定的容量和装填因子（装填因子是一个 0.0 ～ 1.0 之间的数。这个数确定散列表填充到多大比例时就要再散列到一个更大的散列表）。默认的装填因子是 0.75。

API java.util.TreeMap<K,V> 1.2

- TreeMap()

为实现 Comparable 接口的键构造一个空的树映射。

- TreeMap(Comparator<? super K> c)

构造一个树映射，并使用一个指定的比较器对键进行排序。

- TreeMap(Map<? extends K, ? extends V> entries)

构造一个树映射，并增加一个映射中的所有映射条目。

- TreeMap(SortedMap<? extends K, ? extends V> entries)

构造一个树映射，增加一个有序映射中的所有映射条目，并使用与给定有序映射相同的比较器。

API java.util.SortedMap<K, V> 1.2

- Comparator<? super K> comparator()

返回对键进行排序所用的比较器。如果用 Comparable 接口的 compareTo 方法对键进行比较，则返回 null。

- K firstKey()
- K lastKey()

返回映射中的最小或最大键。

9.4.2 更新映射条目

处理映射的一个难点是更新映射条目。正常情况下，可以得到与一个键关联的旧值，更新这个值，再放回更新后的值。不过，必须考虑一个特殊情况，即键第一次出现。下面来看一个例子，考虑使用映射来统计一个单词在文件中出现的频度。看到一个单词（word）时，我们将计数器增 1，如下所示：

```
counts.put(word, counts.get(word) + 1);
```

这是可以的，不过有一种情况除外：就是第一次看到 word 时。在这种情况下，get 会返回 null，因此会出现一个 NullPointerException 异常。

一种简单的补救是使用 getOrDefault 方法：

```
counts.put(word, counts.getOrDefault(word, 0) + 1);
```

另一种方法是首先调用 putIfAbsent 方法。只有当键原先不存在（或者映射到 null）时才放入一个值。

```
counts.putIfAbsent(word, 0);
counts.put(word, counts.get(word) + 1); // now we know that get will succeed
```

不过还可以做得更好。merge 方法可以简化这个常见操作。如果键原先不存在，下面的调用：

```
counts.merge(word, 1, Integer::sum);
```

将把 word 与 1 关联，否则使用 Integer::sum 函数组合原值和 1（也就是将原值与 1 求和）。

API 注释还描述了另外一些更新映射条目的方法，不过这些方法不太常用。

API java.util.Map<K, V> 1.2

- default V merge(K key, V value, BiFunction<? super V,? super V,? extends V> remappingFunction) **8**

  如果 key 与一个非 null 值 v 关联，将函数应用到 v 和 value，将 key 与结果关联，或者如果结果为 null，则删除这个键。否则，将 key 与 value 关联，返回 get(key)。
- default V compute(K key, BiFunction<? super K,? super V,? extends V> remappingFunction) **8**

  将函数应用到 key 和 get(key)。将 key 与结果关联，或者如果结果为 null，则删除这个键。返回 get(key)。
- default V computeIfPresent(K key, BiFunction<? super K,? super V,? extends V> remappingFunction) **8**

  如果 key 与一个非 null 值 v 关联，将函数应用到 key 和 v，将 key 与结果关联，或者如果结果为 null，则删除这个键。返回 get(key)。
- default V computeIfAbsent(K key, Function<? super K,? extends V> mappingFunction) **8**

  将这个函数应用到 key，除非 key 与一个非 null 值关联。将 key 与结果关联，或者如果结果为 null，则删除这个键。返回 get(key)。
- default void replaceAll(BiFunction<? super K,? super V,? extends V> function) **8**

  在所有映射条目上调用这个函数。将键与非 null 结果关联，如果结果为 null，则将相应的键删除。
- default V putIfAbsent(K key, V value) **8**

  如果 key 不存在或者与 null 关联，则将它与 value 关联，并返回 null。否则返回关联的值。

9.4.3 映射视图

集合框架不认为映射本身是一个集合。（其他数据结构框架则认为映射是一个键 / 值对集合，或者是按键索引的值集合。）不过，可以得到映射的视图（view）——实现了 Collection 接

口或某个子接口的对象。

有 3 种视图：键集、值集合（不是一个集）以及键 / 值对集。键和键 / 值对可以构成一个集，因为映射中一个键只能有一个副本。下面的方法：

```
Set<K> keySet()
Collection<V> values()
Set<Map.Entry<K, V>> entrySet()
```

会分别返回这 3 个视图。（映射条目集的元素是实现了 Map.Entry 接口的类的对象。）

需要说明的是，keySet 不是 HashSet 或 TreeSet，而是实现了 Set 接口的另外某个类的对象。Set 接口扩展了 Collection 接口。因此，可以像使用任何集合一样使用 keySet。

例如，可以枚举一个映射的所有键：

```
Set<String> keys = map.keySet();
for (String key : keys)
{
   do something with key
}
```

如果想同时查看键和值，可以通过枚举映射条目来避免查找值。可以使用以下代码：

```
for (Map.Entry<String, Employee> entry : staff.entrySet())
{
   String k = entry.getKey();
   Employee v = entry.getValue();
   do something with k, v
}
```

提示：通过使用 var 声明可以避免笨拙的 Map.Entry。

```
for (var entry : map.entrySet())
{
   do something with entry.getKey(), entry.getValue()
}
```

或者直接使用 forEach 方法：

```
map.forEach((k, v) ->
   {
      do something with k, v
   });
```

如果在键集视图上调用迭代器的 remove 方法，实际上会从映射中删除这个键和与它关联的值。不过，不能向键集视图中添加元素。另外，如果只添加一个键而没有同时添加值也是没有意义的。如果试图调用 add 方法，它会抛出一个 UnsupportedOperationException。映射条目集视图有同样的限制，尽管理论上好像可以增加新的键 / 值对，但实际上并不允许。

API java.util.Map<K, V> 1.2

- Set<Map.Entry<K, V>> entrySet()

  返回 Map.Entry 对象（映射中的键 / 值对）的一个集视图。可以从这个集中删除元素，它们也将从映射中删除，但是不能添加任何元素。

- Set<K> keySet()
  返回映射中所有键的一个集视图。可以从这个集中删除元素，这些键和相关联的值也将从映射中删除，但是不能添加任何元素。
- Collection<V> values()
  返回映射中所有值的一个集合视图。可以从这个集合中删除元素，所删除的值及相应的键也将从映射中删除，不过不能添加任何元素。

API java.util.Map.Entry<K, V> 1.2

- K getKey()
- V getValue()
  返回这个映射条目的键或值。
- V setValue(V newValue)
  将关联映射中的值改为新值，并返回原来的值。
- static <K, V> Map.Entry<K,V> copyOf(Map.Entry<? extends K,? extends V> map) **17**
  生成给定映射条目的一个副本。不同于映射条目集的元素，这个副本不是“活动的”。调用 setValue 不会更新任何映射。

9.4.4 弱散列映射

在集合类库中有几个专用的映射类，我们将在这一节和后面几节中对它们做简要介绍。

设计 WeakHashMap 类是为了解决一个有趣的问题。如果有一个值，它对应的键已经不再在程序中的任何地方使用，将会出现什么情况？假设对某个键的最后一个引用已经消失，那么再没有任何途径可以引用这个值对象了。但是，由于程序中的任何部分都不再有这个键，因此无法从映射中删除这个键 / 值对。为什么垃圾回收器不能删除它呢？删除无用的对象不就是垃圾回收器的工作吗？

遗憾的是，事情没有这么简单。垃圾回收器会跟踪活动的对象。只要映射对象是活动的，其中的所有桶就是活动的，它们就不能被回收。因此，需要由程序负责从长期存活的映射中删除那些无用的值。或者，你可以使用 WeakHashMap。当键的唯一引用来自散列表条目时，这个数据结构将与垃圾回收器合作删除键 / 值对。

下面介绍这种机制的内部工作原理。WeakHashMap 使用弱引用（weak reference）保存键。WeakReference 对象将包含另一个对象的引用，在这里，就是一个散列表键。对于这种类型的对象，垃圾回收器采用一种特殊的方式进行处理。正常情况下，如果垃圾回收器发现某个特定的对象已经没有引用了，就会将其回收。不过，如果这个对象只能由一个 WeakReference 引用，垃圾回收器也会将其回收，但会将引用这个对象的弱引用放入一个队列。WeakHashMap 的操作会定期地检查这个队列，查找新加入的弱引用。一个弱引用进入这个队列意味着这个键不再由任何人使用，并且已经回收。于是，WeakHashMap 将删除相关联的映射条目。